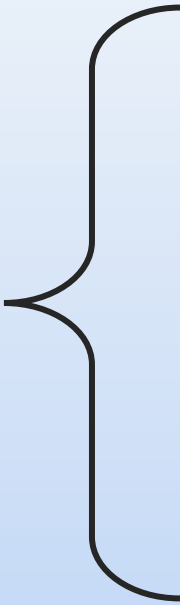
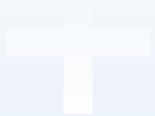


Practical Experiment: **Compare the response speed** **of open-source Large Language Models (LLMs)** **to find the fastest model for real-time applications**

Meeting 1: Environment Initialization & Test AI Agent

Presented by: Mariyan Vasilev Apostolov
Date: 11th of August 2026
Version: 1.0

Agenda



Introduction to Ollama

System Infrastructure

Background Process Logic

System Architecture

Class-Based Implementation

LIVE Demo

Introduction to Ollama

Definition

Ollama is a lightweight, extensible framework for running large language models (LLMs) locally.

Key Features

- **Local Execution:** Run models without internet
- **Multiple Models:** Support for various LLMs
- **API Access:** RESTful API for integration
- **Cross-Platform:** Linux, macOS, Windows (WSL)
- **Model Management:** Pull, update, delete models



4 System Infrastructure

Before executing inference, the environment must be bootstrapped with Linux-level dependencies.

- **apt update:** Refreshes package lists.
- **pciutils:** Inspects hardware for GPU detection.
- **zstd:** Essential for fast model decompression.
- **curl installation:** Fetches the Ollama binary.

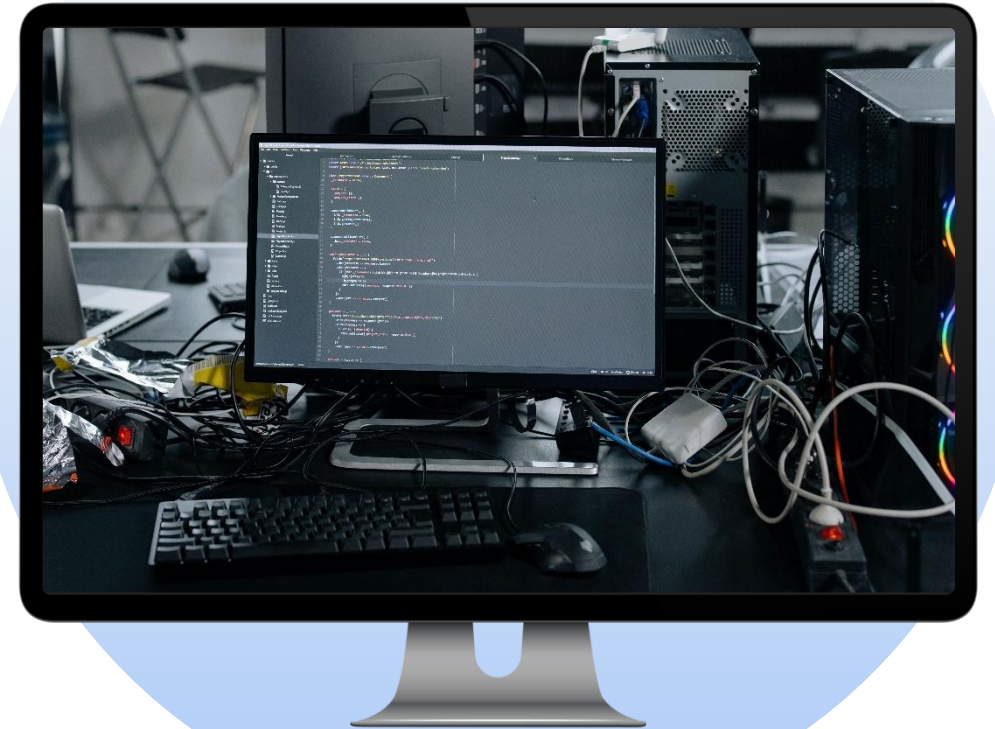


5

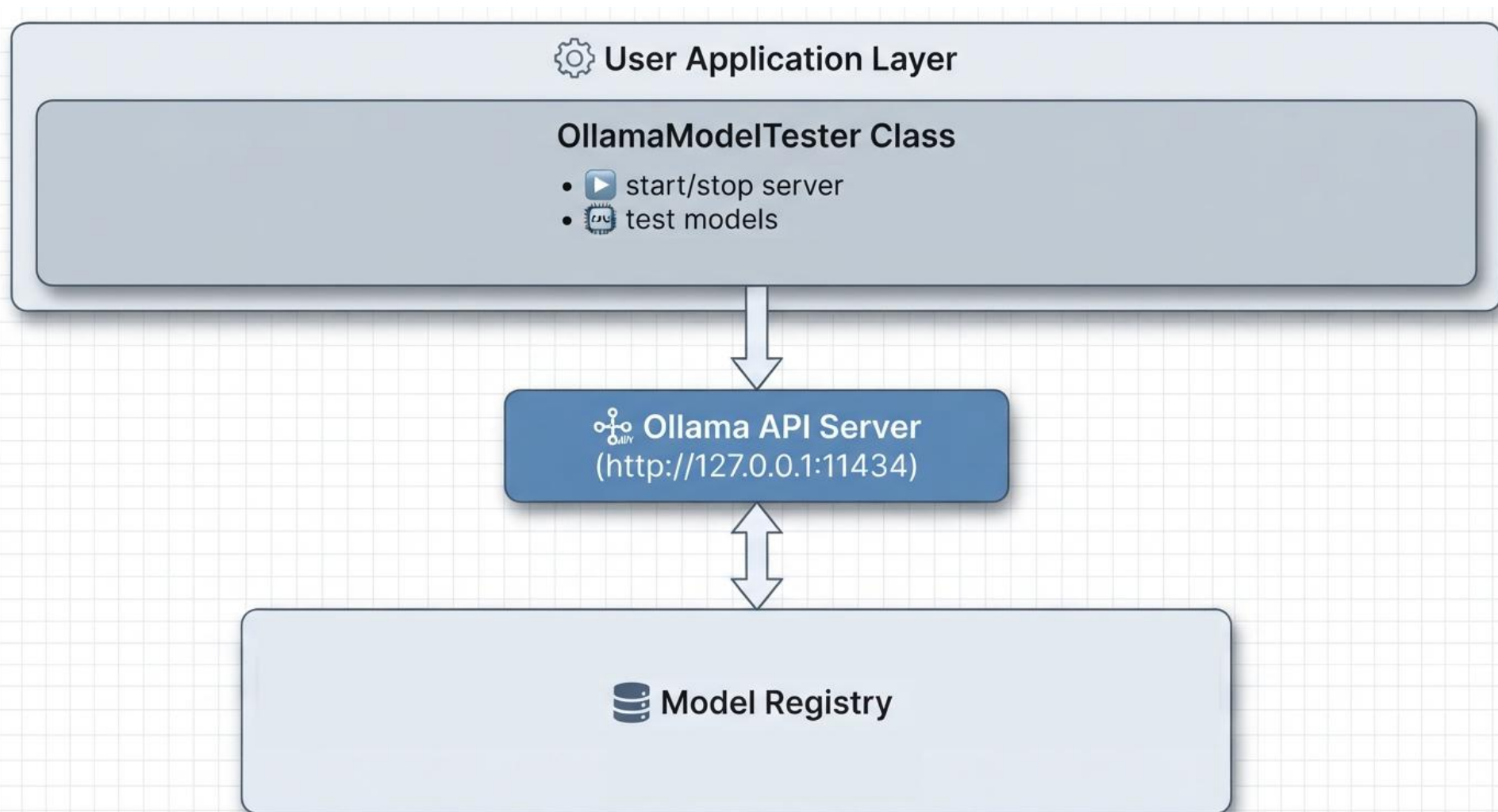
Background Process Logic

Starting the Ollama server programmatically is achieved using `subprocess.Popen`.

- **Environment Variables:** Setting `OLLAMA_HOST` routes requests to `localhost`.
- **Popen:** Spawns 'ollama serve' as a background task.
- **DEVNULL:** Silences logs to prevent terminal clutter.
- **Time Sync:** 10-second sleep ensures server readiness.



6 System Architecture



Class-Based Implementation

class OllamaModelTester:

— Initialization —

def __init__(self, host, port, models):

Server configuration

Model list

Result storage

— Server Management —

def start_server(self): # Start Ollama server

def stop_server(self): # Stop Ollama server

def _check_server_health(self): # Verify server

— Model Management —

def pull_model(self): # Pull a single model

def pull_models(self): # Pull multiple models

— Testing —

def compare_model(self): # Compare one model

def compare_models(self): # Compare multiple models

— Results Processing —

def print_results(self):

— Context Manager —

def __enter__(self): # Auto-start server

def __exit__(self): # Auto-stop server

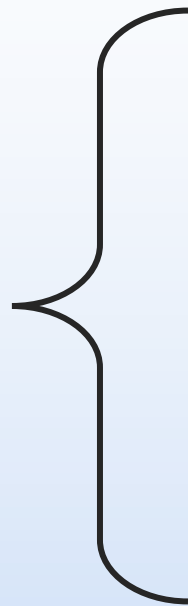




References

- Colab – <https://colab.research.google.com/>
- Subprocess.Popen – <https://docs.python.org/3/library/subprocess.html#subprocess.Popen>
- Subprocess.run – <https://docs.python.org/3/library/subprocess.html#subprocess.run>
- Ollama models – <https://ollama.com/search>
- Ollama generate a response – <https://docs.ollama.com/api/generate>
- Nest_asyncio library for Python – <https://pypi.org/project/nest-asyncio/>
- Requests library for Python – <https://requests.readthedocs.io/en/latest/>

Thank you



Mariyan Vasilev Apostolov

mariyan.apostolov89@gmail.com